

The UM6P Vanguard Center

AI Tutor Engineering Internship

ENGINEERING WORK PACKAGE Final

Ontology-Gated Instance Generation for Learning-Space Items

AI-TUTOR-WP-003

Owner	Islam Bouikiri
Mentor	Jalal Maaouni, Asmae Afifi
Start	2026-08-24
Target	2026-08-31
Review cadence	Daily checkpoint
Repository	Your private Github Repo
Submission	Submit the artifact on Github.

Executive brief

Generate candidate learning instances for a given Learning Space Theory item. Use the lecture ontology to define the allowed knowledge scope. Build the gate with deterministic graph code. Use the LLM only after the gate is ready. Validate each generated instance against the gate before you accept it.

1 Objective

Objective

Build a small pipeline that takes one learning item and one or more ontology anchors. Compute the allowed ontology scope. Generate candidate instances only from this scope. Validate the generated instances. Reject unsupported or out-of-scope content.

2 Context

WP-003 used the ontology as a source for factual answers. WP-004 uses the same ontology as a generation boundary.

Use this sequence:

```

1 LST item
2   -> ontology anchor
3   -> deterministic scope gate
4   -> allowed ontology evidence
5   -> structured LLM generation
6   -> validation
7   -> candidate instance

```

3 Source material

Use these inputs:

- The OWL ontology from WP-003.
- The deterministic graph service from WP-003.
- A small set of mentor-approved LST items.

Treat the ontology as the source of truth for lecture knowledge. Treat the item definition as the source of truth for the target skill or concept.

4 Reading and research task

Complete the reading task before you implement the generator. Use primary sources and official documentation when possible.

Theme	Material	Priority
Learning Space Theory	Review the terms item , instance , knowledge state , and surmise relation . Focus on the difference between an item and an instance.	Required
Knowledge-space foundations	Read selected sections from Doignon and Falmagne on knowledge spaces and learning spaces. Focus on items, states, and prerequisite structure.	Required
Structured generation	Pydantic Models. Study typed models, validation, optional fields, and validation errors.	Required
LangChain structured output	LangChain Structured Output. Study schema-based output and validation.	Required
Knowledge graph grounding	Review the RDF, OWL, SPARQL, and RDFLib notes from WP-003. Focus on graph evidence and relation semantics.	Required
LLM evaluation	Research grounded generation, post-generation validation, hallucination, and constraint checking.	Required

4.1 Research topics

Research and explain these terms:

- Learning item and learning instance.
- Knowledge state and surmise relation.
- Ontology anchor.

- Graph neighborhood and traversal depth.
- Allowed entity and allowed relation.
- Grounded generation.
- Structured output.
- Deterministic validation.
- Pedagogical validation.
- Provenance.
- Distractor.
- Hallucination and scope leakage.

4.2 Reading artifact

Create `wp4_reading_notes.md`. Use your own words.

Use this structure:

1. Explain the difference between an LST item and an instance.
2. Explain why one item can have many instances.
3. Explain why ontology grounding does not guarantee pedagogical quality.
4. Compare prompt-only constraints with deterministic constraints.
5. Explain how graph relations can support different question types.
6. Explain how structured output helps validation.
7. Give three examples of scope leakage.
8. Give three examples of valid ontology-grounded instances.
9. Record open questions for the review meeting.

Be ready to explain each term without notes.

5 Input contract

Start with a small item file. Use a fixed ontology anchor for each item.

Example:

```
1 {  
2   "item_id": "COMP101-L10-I017",  
3   "title": "Explain the purpose of encapsulation",  
4   "description": "Explain why object state is hidden behind a controlled interface.",  
5   "ontology_entities": ["Encapsulation"]  
6 }
```

Do not infer the anchor in this work package. Add the anchor by hand before generation.

Use at least five mentor-approved items from one lecture.

6 Scope gate

The scope gate must be deterministic. Do not ask the LLM to decide its own scope.

For each item, compute an allowed subgraph from the ontology anchor. The gate must record:

- Anchor entities.
- Allowed entities.
- Allowed relations.
- Traversal depth.
- Evidence triples.

Use a structure such as:

```

1 {
2   "item_id": "COMP101-L10-I017",
3   "anchors": ["Encapsulation"],
4   "max_depth": 1,
5   "allowed_entities": [
6     "Encapsulation",
7     "Class",
8     "Data Hiding"
9   ],
10  "allowed_relations": [
11    "dependsOn",
12    "enables"
13  ],
14  "evidence": [
15    ["Encapsulation", "dependsOn", "Class"],
16    ["Encapsulation", "enables", "Data Hiding"]
17  ]
18 }
```

Do not include every nearby graph edge by default. Define the relation policy explicitly.

7 Relation policy

Study which ontology relations can support useful instances. Do not treat all edges as equivalent.

Use a table such as this one in the implementation notes:

Relation role	Possible instance use
Definition or description	Concept explanation or identification
Dependency or prerequisite	Prerequisite question
Contrast	Comparison or discrimination question
Implementation or mechanism	Mechanism or example question
Error relation	Debugging or error-identification question
Produced type or output relation	Output prediction question

Record which ontology relations you allow for each item. Explain each choice.

8 Instance types

Support only these instance types in WP-004:

- Short answer.
- True or false.
- Multiple choice.

Do not generate code questions in this work package.

For each item, generate at least:

- Three short-answer instances.
- Three true-or-false instances.
- Three multiple-choice instances.

With five items, this gives at least 45 candidate instances.

9 Output schema

Use structured output. Do not accept free-form generation as the internal result.

Define a Pydantic model. Include at least these fields:

```

1 class CandidateInstance(BaseModel):
2     item_id: str
3     instance_type: Literal["short_answer", "true_false", "mcq"]
4     prompt: str
5     answer_key: str
6     choices: list[str] | None = None
7     ontology_entities_used: list[str]
8     ontology_evidence: list[EvidenceTriple]
9     generation_notes: str | None = None

```

Use the same field names for all generated records. Reject malformed output.

10 Work sequence

Follow the sequence below. Do not start the next step before the current step works.

10.1 Step 1: Reuse and refactor the WP-003 graph service

Reuse the deterministic ontology code from WP-003. Do not copy graph logic into the generator.

If necessary, refactor the service so that the ontology path is an input.

Prefer this design:

```

1 service = GraphService("ontology/comp101_L10.owl")

```

10.2 Step 2: Build the deterministic scope gate

Create `scope_gate.py`.

Implement functions such as:

```
1 def build_scope(  
2     item: Item,  
3     max_depth: int = 1,  
4 ) -> OntologyScope:  
5     ...  
6  
7 def is_entity_allowed(scope: OntologyScope, entity_id: str) -> bool:  
8     ...  
9  
10 def is_relation_allowed(scope: OntologyScope, relation: str) -> bool:  
11     ...  
12  
13 def is_evidence_allowed(  
14     scope: OntologyScope,  
15     triple: EvidenceTriple,  
16 ) -> bool:  
17     ...
```

Test these functions without an LLM.

10.3 Step 4: Define the generation schema

Create `schemas.py`. Define models for:

- Item.
- Ontology scope.
- Evidence triple.
- Candidate instance.
- Validation result.

Use validation rules. For example, require MCQ choices only for MCQ instances.

10.4 Step 5: Build the generator

Create `generator.py`.

Give the model only the information that it needs:

- Item identifier.
- Item title.
- Item description.
- Allowed ontology entities.
- Allowed ontology relations.
- Evidence triples.
- Requested instance type.

Do not give the full ontology to the model.

Tell the model to return the structured instance schema only.

10.5 Step 6: Validate ontology support

Create `validator.py`.

Implement deterministic checks first.

Check at least these conditions:

- Every used entity is allowed.
- Every used relation is allowed.
- Every cited evidence triple exists in the allowed scope.
- The answer key has ontology support.
- The item identifier exists.
- The output schema is valid.

Reject the candidate if a required check fails.

10.6 Step 7: Validate pedagogical quality

Ontology validity is not enough. Check pedagogical quality separately.

Review these conditions:

- The instance targets the selected item.
- The instance is not trivial.
- The wording is clear.
- The answer is unambiguous.
- The difficulty is reasonable for the item.
- MCQ distractors are plausible but incorrect.

You can use an LLM reviewer for this step. Mark this result as a model judgment. Do not present it as deterministic proof.

Include a final human-review field.

10.7 Step 8: Generate the candidate set

Generate at least 45 candidate instances.

Store all results. Keep rejected candidates. Do not delete failures from the dataset.

For each candidate, record:

- Generation input.
- Generated instance.
- Ontology validation result.
- Pedagogical validation result.
- Final human decision.
- Rejection reason, when applicable.

10.8 Step 9: Test failure cases

Add deliberate failure cases.

Include at least these cases:

- An out-of-scope concept appears in the prompt.
- An out-of-scope concept appears in the answer.
- The evidence triple does not exist.
- The instance is valid but tests the wrong item.
- The instance is trivial.
- An MCQ has two correct answers.
- An MCQ distractor is obviously invalid.
- The model returns malformed structured output.

The validator must reject the deterministic failures.

11 Evaluation

Create `evaluation_report.md`.

Report at least these metrics:

- Total generated candidates.
- Ontology pass rate.
- Scope violation rate.
- Unsupported-answer rate.
- Item-alignment rate.
- Human acceptance rate.
- Duplicate or near-duplicate rate.

Separate deterministic results from model judgments.

For each rejected candidate, record the main reason.

Do not report only aggregate scores. Include representative failures.

12 Review questions

Be ready to answer these questions during the final review:

1. What is the difference between an item and an instance?
2. Why do you use a manual ontology anchor in this work package?
3. What makes the scope gate deterministic?
4. Why is a prompt such as “stay inside the ontology” not a sufficient gate?

5. How do you prove that an evidence triple is allowed?
6. Why can an ontology-valid instance still be a bad assessment instance?
7. How do you detect an instance that tests the wrong item?
8. How do ontology relations affect the type of instance that you generate?
9. Why are MCQ distractors difficult to generate safely?
10. What information must you save for provenance?

13 Acceptance criteria

Acceptance criteria

The work is accepted when all criteria below are true.

- ✓ The intern can explain the difference between an LST item and an instance.
- ✓ The input set contains at least five mentor-approved items.
- ✓ Each item has at least one explicit ontology anchor.
- ✓ The scope gate is deterministic and has unit tests.
- ✓ The generator receives only the allowed graph evidence.
- ✓ The generator returns structured output that matches the schema.
- ✓ The deterministic validator rejects out-of-scope entities and relations.
- ✓ The deterministic validator rejects unsupported evidence triples.
- ✓ The implementation separates ontology validation from pedagogical validation.
- ✓ The candidate set contains at least 45 generated instances.
- ✓ The dataset keeps rejected candidates and rejection reasons.
- ✓ The evaluation report separates deterministic results from model judgments.
- ✓ The report includes the human acceptance rate.
- ✓ The report includes representative failure cases.